

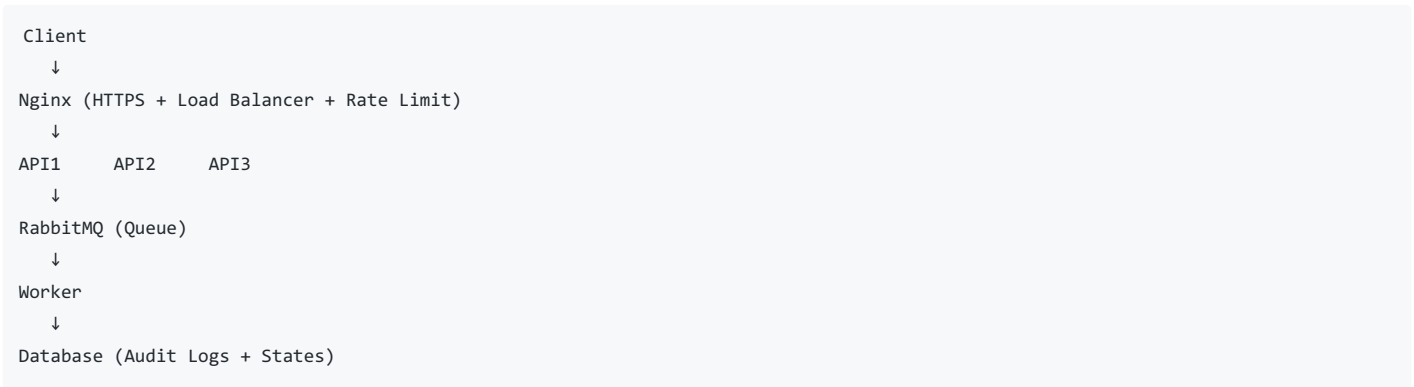
Assignment: Implementation of a Secure Distributed System with Observability and Security Analysis

Objective

In this assignment, you are required to implement a secure distributed system that demonstrates:

- Load balancing
 - HTTPS communication
 - Authentication between services
 - Asynchronous processing using a message broker
 - Rate limiting
 - Persistent audit logging using a database
 - Full request state tracking
 - Security analysis using a Man-in-the-Middle (MITM) simulation
-

System Architecture



System Components

1. Nginx (Gateway Layer)

You must configure Nginx to:

- Act as a reverse proxy
 - Perform load balancing across API instances
 - Enforce rate limiting
 - Enable HTTPS communication
 - Redirect HTTP traffic to HTTPS
-

2. API Service

You must implement an API service that:

- Exposes an endpoint (e.g., /task)
 - Receives requests from the client through Nginx
 - Validates requests using JWT authentication
 - Generates a unique Request ID (UUID) for each request
 - Logs all events into the database
 - Sends tasks to RabbitMQ
-

3. Multiple API Instances

You must run at least three API instances:

- api1
- api2
- api3

Each instance must:

- Handle requests independently

- Identify itself in the response
-

4. RabbitMQ (Message Broker)

You must configure RabbitMQ to:

- Receive tasks from the API
 - Store them in a queue
 - Deliver them to the worker service
-

5. Worker Service

You must implement a worker service that:

- Consumes messages from RabbitMQ
 - Verifies incoming data (service identity validation)
 - Processes tasks
 - Logs all processing events into the database
-

6. Database (Containerized)

You must add a database service using Docker (e.g., PostgreSQL, MySQL, or MongoDB).

The database must:

- Run as a separate container
 - Store all audit logs
 - Store all request states
-

Audit Logging Requirements

Each log entry must include:

- Timestamp
 - Service name
 - Request ID
 - Action performed
 - Status (success / failure)
 - Source (client or service)
-

State Tracking Requirements

Required States

- RECEIVED
- AUTHENTICATED
- QUEUED
- CONSUMED
- PROCESSED
- FAILED

Requirements

- Each request must have a unique Request ID
 - The same Request ID must be passed across all services
 - Every state transition must be stored in the database with timestamp
-

Functional Flow

1. Client sends HTTPS request
2. Nginx forwards request to one API instance
3. API validates JWT token
4. API generates Request ID
5. API logs RECEIVED and AUTHENTICATED states
6. API sends task to RabbitMQ
7. API logs QUEUED state
8. Worker consumes task
9. Worker logs CONSUMED state
10. Worker processes task
11. Worker logs PROCESSED state

MITM Attack Simulation (Wireshark)

Step 1: HTTP Mode

- Disable HTTPS in Nginx
- Run the system using HTTP

Step 2: Capture Traffic

Use Wireshark and capture HTTP traffic.

You must demonstrate:

- Visibility of request headers
- JWT token exposed in plain text
- Readable request payload

Step 3: HTTPS Mode

- Enable HTTPS using Nginx
- Use a local certificate

Step 4: Capture Traffic Again

You must demonstrate:

- Encrypted packets
- No readable payload
- No visible token

Testing Requirements

- Normal request must work
- Load balancing must distribute requests
- Unauthorized requests must fail
- Rate limiting must block excessive requests
- Messages must flow through RabbitMQ
- Logs must be stored in database

Deliverables

1. Full source code
2. docker-compose file
3. Database schema
4. Screenshots:
 - Load balancing
 - RabbitMQ
 - Database logs
 - Wireshark (HTTP & HTTPS)
5. Short report
6. GitHub Repo

Team Requirements

- Each team must consist of **4 to 6 members**
 - Work must be collaborative across implementation, testing, and documentation
-

Discussion Session

Each team must be prepared for a discussion session. During the discussion, you may be asked to:

- Explain the system architecture
 - Justify design decisions
 - Demonstrate security mechanisms
 - Explain request tracking across services
 - Compare HTTP vs HTTPS using your Wireshark results
-

Important Note

All team members must be able to explain the system. Individual understanding will be considered during evaluation.

Final Instruction

This assignment represents a complete secure distributed system with observability and security validation.